PhishGuard: An End-to-End Machine Learning System for Phishing Email Detection

AHMED FOUATIH Hamza Faiz*, GOUICEM Islem*, GADIRI Amina*,
CHERGUI Mohamed Bahae Eddine*, AMEDDAH Mohamed*
*Department of Intelligent Systems Engineering,
ENSIA, Algiers

***Abstract*—Phishing emails remain one of the most prevalent vectors of cybercrime, responsible for credential theft, financial fraud, and large-scale data breaches. This paper presents *PhishGuard*, an end-to-end machine learning system for automatic phishing email classification. Starting from a raw corpus of 18,650 labeled emails, we designed a rigorous data cleaning pipeline, engineered a 8,022-dimensional feature representation combining structural heuristics, psycholinguistic lexicons, stylometric measures, and TF-IDF n-gram features, and systematically benchmarked seven classifiers. The best-performing model — a calibrated Linear Support Vector Machine — achieved a test ROC-AUC of 0.9985, F1-score of 0.9825, and an error rate of only 1.32% on 2,573 held-out samples, with 18 false positives and 16 false negatives. The trained model was deployed as a RESTful API using FastAPI and integrated into a browser extension providing real-time phishing detection within Gmail and Outlook.**

***Index Terms*—phishing detection, email classification, natural language processing, TF-IDF, support vector machine, feature engineering, FastAPI, browser extension**

I. Introduction

Email remains the dominant communication channel in professional and personal settings, making it a primary attack surface for malicious actors. Phishing — the practice of impersonating trusted entities to deceive recipients into revealing sensitive information — accounts for the majority of cyberattacks globally [1]. Traditional rule-based spam filters are increasingly inadequate against sophisticated campaigns that closely mimic legitimate communication styles, using correct grammar, familiar brand imagery, and plausible narratives.

Machine learning offers a data-driven alternative: rather than manually curating detection rules, a classifier learns the statistical patterns that distinguish phishing from legitimate email directly from labeled examples. This work explores that approach end-to-end, from raw data ingestion to a deployed, browser-integrated detection system.

The contributions of this work are fourfold:

- A reproducible multi-stage cleaning pipeline with quality gates and full traceability via machine-readable reports.
- A domain-aware feature engineering strategy combining 22 handcrafted features with 8,000 TF-IDF features (8,022 total) with strict zero-leakage discipline.
- A systematic benchmark of seven classifiers with hyperparameter tuning, threshold analysis, and full held-out test evaluation including calibration and error analysis.
- A production-ready FastAPI deployment integrated with a Chrome browser extension for real-time inference inside email clients.

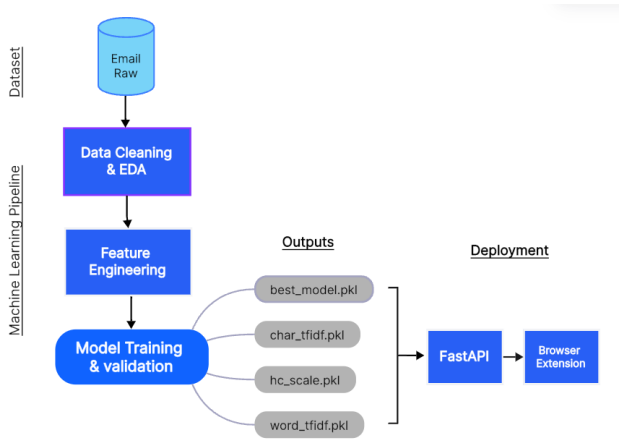

Fig. 1. End-to-end architecture of the PhishGuard system.

II. Related Work

Early approaches to phishing detection relied on blacklists and URL-based heuristics. Fette et al. [1] were among the first to frame phishing detection as a supervised classification problem, achieving strong results with a Random Forest on manually crafted email features. Subsequent work extended this with TF-IDF representations and SVMs [2], demonstrating that lexical content carries substantial discriminative power.

More recently, deep learning approaches using CNNs and LSTMs have been explored, and pre-trained transformer models such as BERT have shown promising results on email classification tasks. However, classical methods remain highly competitive on constrained datasets and offer significant advantages in inference latency and interpretability — both critical in a real-time deployment context. This work deliberately combines handcrafted domain features with TF-IDF representations to exploit both structured security knowledge and the generalization power of learned representations [3], [6].

III. Dataset and Data Cleaning

A. Dataset

The raw dataset consisted of **18,650 labeled emails** in CSV format, with two fields: `Email Text` (raw content) and `Email Type` (*Safe Email* or *Phishing Email*).

B. Cleaning Pipeline

A multi-stage cleaning pipeline was applied with the following quality gates:

1) **Label standardization:** Labels were mapped to binary integers (0 = safe, 1 = phishing); rows with unrecognized labels were removed.
2) **Minimum text quality:** Samples with fewer than 20 characters or 3 words after cleaning were discarded as low-signal.
3) **Deduplication:** Exact duplicate (text, label) pairs were removed, keeping the first occurrence.
4) **Conflict removal:** Samples where identical cleaned text was assigned to both classes were removed to prevent contradictory training signal.

Table I summarizes the outcome of each stage. In total, 1,498 samples (8.03% of the raw corpus) were removed.

TABLE I
Data Cleaning Report

Stage	Removed	Remaining
Raw dataset	—	18,650
Invalid labels	0	18,650
Low-quality text	567	18,083
Exact duplicates	931	17,152
Label conflicts	0	17,152
Final clean dataset	**1,498**	**17,152**

The final dataset contains 10,693 safe emails (62.3%) and 6,459 phishing emails (37.7%), reflecting a moderate class imbalance addressed during modeling via class weighting.

C. Text Normalization

Each email was normalized through eight sequential steps: HTML entity decoding and tag removal, Unicode normalization (NFKC), control character removal, lowercasing, and replacement of URLs, email addresses, and numbers with semantic placeholder tokens (`<url>`, `<email>`, `<num>`). Using placeholder tokens rather than deletion preserves the *presence* of these elements as classification signals while removing values too sparse to generalize.

D. Train / Validation / Test Split

A stratified 70/15/15 split was applied, preserving the 62/38 class ratio across all three partitions. The test set (2,573 samples) was withheld entirely until final evaluation in Phase 5.

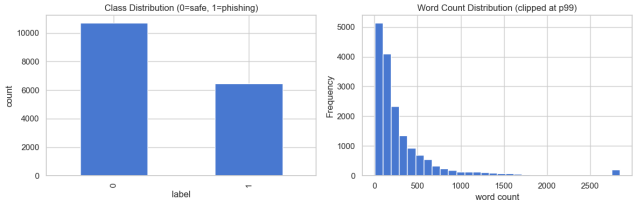

Fig. 2. Class distribution (left) and word count distribution (right) after cleaning.

IV. Exploratory Data Analysis

Exploratory analysis was conducted exclusively on the training split to prevent information leakage into feature design decisions. Three categories of divergence were identified.

Structural divergence. Phishing emails exhibit significantly higher `<url>` and `<num>` placeholder density compared to safe emails, consistent with the core phishing mechanism of redirecting victims to fraudulent external resources via embedded hyperlinks.

Lexical divergence. Safe emails are rooted in institutional discourse (prominent terms: *enron*, *university*, *conference*), while phishing emails are dominated by financial and urgency vocabulary (*free*, *money*, *verify*, *click*, *prize*).

Stylometric divergence. Phishing emails exhibit a measurably lower Type-Token Ratio (TTR), confirming their formulaic, template-driven nature relative to legitimate correspondence.

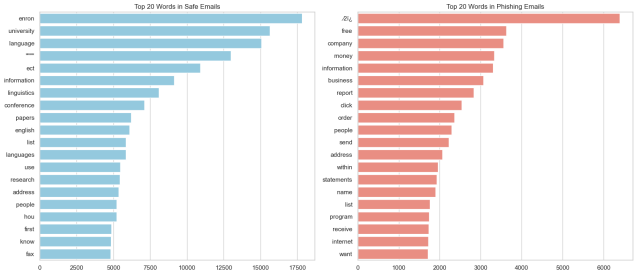

Fig. 3. Average placeholder frequency per email by class (left) and top 20 discriminative words per class (right).

V. Feature Engineering

Based on EDA findings, features were engineered across four groups yielding **8,022 features** per email, as summarized in Table II.

TABLE II
ENGINEERED FEATURE GROUPS

Group	Key Features	#
Structural	url_count, url_density, num_count, word_count, char_count, avg_word_length, punct_count, ...	11
Urgency & Sentiment	urgency_count, financial_count, action_count, threat_count, caps_ratio, urgency_density	6
Stylometric	unique_word_ratio, sentence_count, avg_sentence_length, placeholder_ratio, digit_char_ratio	5
Word TF-IDF	Unigrams + bigrams, sublinear TF, min_df=3	5,000
Char TF-IDF	3–5 char n-grams (char_wb), min_df=5	3,000
Total		**8,022**

A. Structural and Heuristic Features

Eleven features encode observable structural signals. URL count and URL density (normalized by word count) are the dominant individual discriminators. Exclamation and question mark counts from the raw text capture punctuation-based urgency signals lost after normalization.

B. Urgency and Sentiment Lexicon Features

Four curated lexicons — urgency, financial, action, and threat — were constructed from domain knowledge of phishing manipulation patterns. Term occurrences are counted and normalized by email length to produce length-invariant density features. The uppercase character ratio (`caps_ratio`) is computed on the raw text, as cleaning lowercases all content.

C. Stylometric Features

The Type-Token Ratio (`unique_word_ratio`) quantifies vocabulary richness: a low TTR indicates repetitive, formulaic text — a hallmark of template-driven phishing. Average sentence length and sentence count capture structural complexity differences between classes.

D. TF-IDF Features

Two TF-IDF vectorizers were fitted exclusively on the training set. A word-level vectorizer with unigrams and bigrams (max 5,000 features, sublinear TF scaling, `min_df`=3) captures discriminative lexical content. A character-level vectorizer with 3–5 character n-grams within word boundaries (`char_wb`, max 3,000 features, `min_df`=5) detects sub-word morphological patterns and obfuscated keywords (e.g., *acc0unt*, *veri-fy*) invisible to word tokenization. A custom token pattern ensures that placeholder tokens (`<url>`, `<email>`, `<num>`) are treated as first-class vocabulary items by the word vectorizer.

E. Scaling and Leakage Prevention

Handcrafted features were scaled with a `MinMaxScaler` fitted on the training set only. All transformations were applied to validation and test sets via `transform()` only — never `fit_transform()` — guaranteeing zero data leakage across all experimental phases.

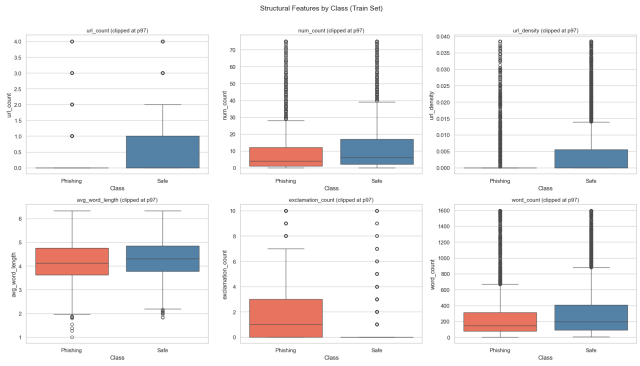

Fig. 4. Structural feature distributions by class (top) and top TF-IDF discriminative tokens (bottom).

VI. MODELING

A. Experimental Protocol

Seven classifiers were trained and evaluated on the validation set using ROC-AUC as the primary metric, which is robust to class imbalance and threshold choice [6]. Secondary metrics include F1-score, precision, recall, and accuracy at $\tau = 0.5$. All models using the full feature matrix were trained with `class_weight="balanced"` to compensate for the 62/38 imbalance. Hyperparameters were tuned via grid search on the validation set; the test set was withheld entirely from model selection.

B. Models Evaluated

1) **Logistic Regression [3]:** L2-regularized, solver `saga`, $C \in \{0.1, 1.0, 5.0, 10.0\}$.
2) **Multinomial Naive Bayes:** Laplace smoothing $\alpha \in \{0.01, 0.05, 0.1, 0.5, 1.0, 2.0\}$.
3) **Linear SVM [6]:** `LinearSVC` wrapped with `CalibratedClassifierCV` (cv=3); $C \in \{0.01, 0.1, 0.5, 1.0, 5.0\}$.
4) **Random Forest (HC only):** 300 estimators on 22 handcrafted features — ablation study.
5) **XGBoost [4]:** Full sparse matrix, early stopping on validation AUC, `scale_pos_weight` = inverse class ratio.
6) **Neural Network (MLP):** Two hidden layers $(512 \to 256)$, ReLU, Adam, early stopping.
7) **Voting Ensemble (LR + SVM):** Soft voting over class probabilities of Logistic Regression and Linear SVM.

C. Validation Results

TABLE III
MODEL LEADERBOARD — VALIDATION SET (SORTED BY ROC-AUC)

Model	AUC	F1	P	R	Acc
Linear SVM	**.9985**	**.9825**	.9815	.9835	.9868
Voting (LR+SVM)	.9985	.9825	.9815	.9835	.9868
MLP	.9980	.9794	.9784	.9804	.9845
Logistic Regression	.9979	.9810	.9765	.9856	.9856
Naive Bayes	.9936	.9519	.9544	.9494	.9639
XGBoost	.9935	.9442	.9198	.9701	.9569
RF (HC only)	.9454	.8305	.8531	.8091	.8756

D. Analysis of Validation Results

Linear models dominate. Both Logistic Regression and Linear SVM achieve ROC-AUC above 0.997, outperforming XGBoost (0.9935) and the MLP (0.9980). This is consistent with established findings on high-dimensional sparse text classification, where linear decision boundaries are often sufficient [2].

Handcrafted features alone are insufficient. The Random Forest ablation (22 features, AUC 0.9454, F1 0.8305) confirms that lexical content captured by TF-IDF is the primary discriminative signal, while handcrafted features serve as a valuable complement.

The voting ensemble provides no marginal gain. Soft voting over LR and SVM matches but does not exceed the standalone SVM, indicating overlapping error patterns between the two linear models on this dataset.

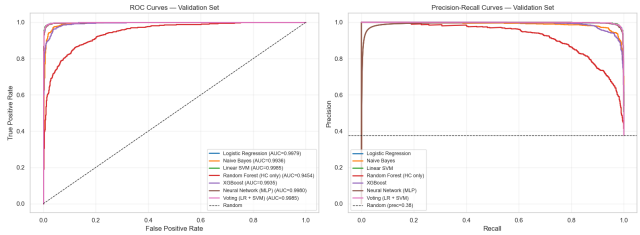

Fig. 5. ROC curves on the validation set for all trained models.

E. Threshold Analysis on Validation Set

A sweep over $\tau \in [0.05, 0.95]$ identified two optimal operating points:

- **Max F1:** $\tau = 0.60$, F1 $= 0.9829$.
- **High recall** (Recall ≥ 0.95): $\tau = 0.81$, Precision $= 0.9957$.

In a security context, the high-recall configuration is operationally preferable: the cost of a missed phishing email (false negative) typically exceeds the cost of a false alarm.

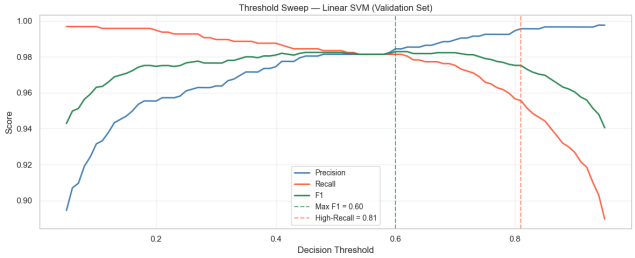

Fig. 6. Precision, Recall, and F1-score as a function of decision threshold τ for the Linear SVM (validation set).

VII. EVALUATION ON HELD-OUT TEST SET

A. Test Metrics at Default Threshold

The selected Linear SVM was evaluated on the held-out test set of **2,573 samples** — unseen throughout all modeling and tuning decisions. Table IV reports performance at $\tau = 0.5$.

TABLE IV
TEST SET PERFORMANCE — LINEAR SVM ($\tau = 0.5$)

Metric	Value
ROC-AUC	**0.9985**
F1-score	**0.9825**
Precision	0.9815
Recall	0.9835
Accuracy	0.9868
Brier Score	0.0109
Total errors	34 / 2,573 (1.32%)
False positives	18
False negatives	16

Test metrics are **identical to validation**, confirming that the model generalizes well and no overfitting occurred during model selection. The Brier score of 0.0109 — close to 0 (perfect) — confirms excellent probability calibration: the model's confidence scores reliably reflect true classification uncertainty.

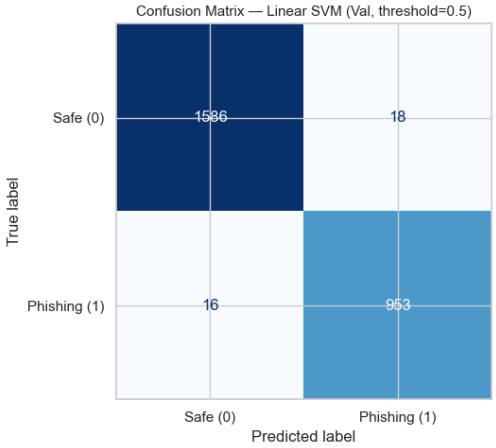

Fig. 7. Confusion matrix of the Linear SVM on the held-out test set ($\tau = 0.5$). Only 34 out of 2,573 samples were misclassified.

B. Threshold Transfer from Validation to Test

The validation-derived operating thresholds were applied directly to the test set to verify deployment reliability. Table V reports the results.

TABLE V
THRESHOLD TRANSFER — VALIDATION THRESHOLDS ON TEST SET

τ	F1	Prec.	Rec.	Acc.
0.50 (default)	0.9825	0.9815	0.9835	0.9868
0.60 (max F1)	0.9808	0.9844	0.9773	0.9856
0.81 (high rec.)	0.9711	0.9893	0.9536	0.9786

All three thresholds transfer cleanly with no ordering degradation: higher τ consistently trades recall for precision as expected. The diagnostic best-F1 threshold on the test set is $\tau = 0.48$ (F1 = 0.9825), confirming that the default $\tau = 0.5$ is already near-optimal.

C. ROC, Precision-Recall, and Calibration Curves

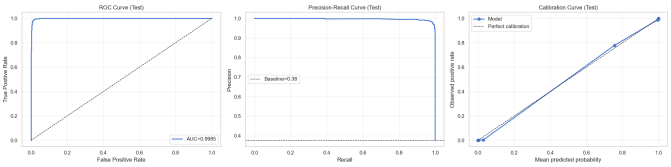

Fig. 8. Test set evaluation curves for the Linear SVM. Left: ROC curve (AUC = 0.9985). Center: Precision-Recall curve. Right: Calibration reliability diagram showing predicted probabilities vs. observed positive rates (Brier = 0.0109).

The calibration reliability diagram (right panel) shows that the model's predicted probabilities closely follow the diagonal — indicating that predicted probabilities are generally consistent with observed phishing frequencies. This property is essential for the browser extension, where users interpret the displayed confidence score to make decisions.

D. Error Analysis by Email Length

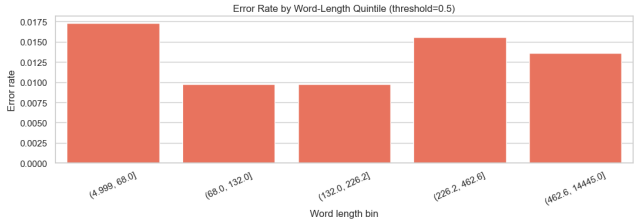

Fig. 9. Error rate per word-length quintile on the test set ($\tau = 0.5$). Shorter emails are harder to classify correctly.

The error-by-length analysis reveals that misclassifications are disproportionately concentrated in **short emails** (lowest word-count quintile). Short emails provide little lexical context for TF-IDF features, making the model rely more heavily on structural signals that are less discriminative in isolation.

E. Qualitative Error Analysis

Of the 34 misclassified samples, manual inspection reveals two dominant failure patterns:

False positives (18 cases) — legitimate promotional or transactional emails flagged as phishing. These emails often contain multiple URLs, discounts, numbers, and action-oriented terms such as *confirm* or *verify*, which closely resemble phishing structures despite being benign.

False negatives (16 cases) — phishing emails classified as safe. Many of these emails avoid typical phishing indicators such as urgency, financial vocabulary, or suspicious links, instead using conversational or newsletter-like language that appears legitimate.

Overall, the results suggest that the classifier relies heavily on surface-level lexical patterns. Incorporating sender-level signals (domain reputation, SPF/DKIM headers) and richer contextual features could improve robustness and reduce both error types.

VIII. System Deployment

A. RESTful API

The trained Linear SVM was deployed as a RESTful API using FastAPI [5]. Three endpoints are exposed: `GET /` (status), `GET /health` (liveness probe), and `POST /predict` (inference). The predict endpoint accepts `{"text": "..."}` and returns the binary label, human-readable verdict, phishing probability, and model name.

The inference pipeline replicates training preprocessing exactly: `clean_text()` → `extract_handcrafted()` → `MinMaxScaler.transform()` → `TfidfVectorizer.transform()` ×2 → `hstack` → `predict_proba()`. All fitted objects are loaded once at server startup and reused across requests, keeping per-email latency under one millisecond on CPU. CORS headers explicitly permit requests from Gmail, Outlook, and Chrome extension origins.

B. Browser Extension

A Chrome browser extension intercepts email body content and forwards it to the local API via `POST /predict`, using `JSON.stringify()` to correctly escape newlines, quotes, and control characters present in real email bodies. The verdict and confidence score are displayed inline within the email client interface.

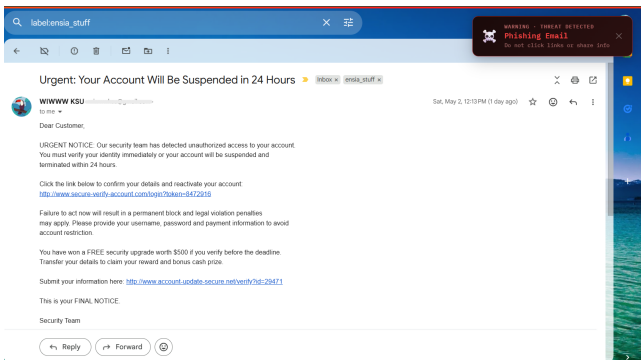

Fig. 10. PhishGuard browser extension displaying a real-time phishing verdict inside Gmail.

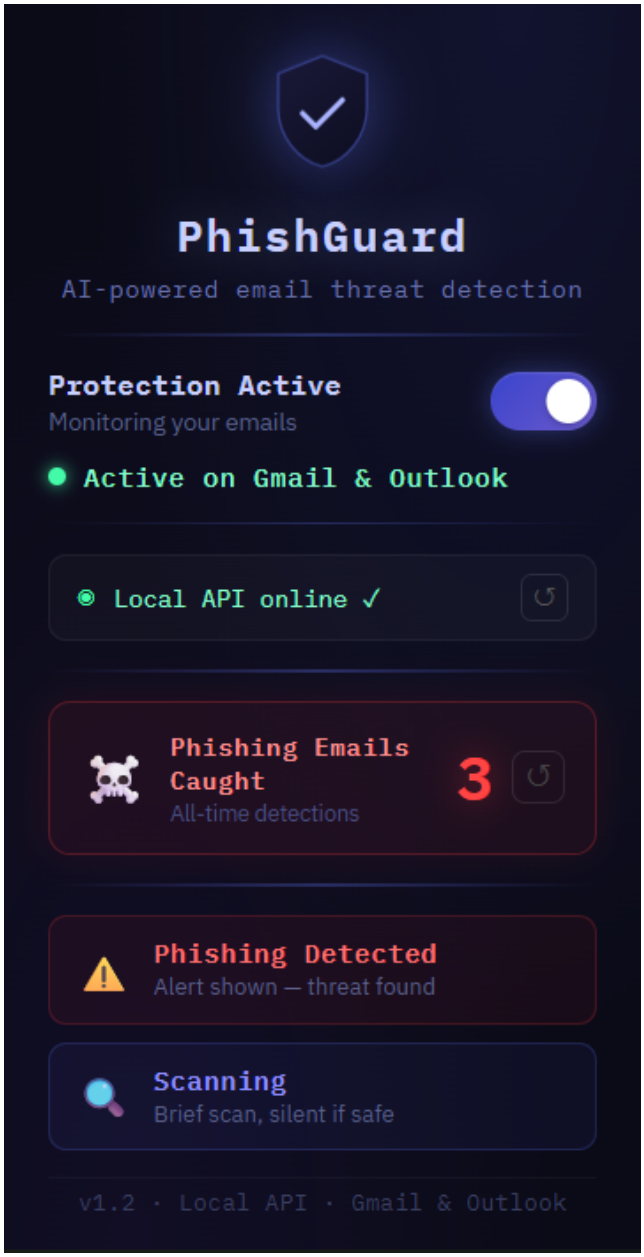

Fig. 11. PhishGuard browser extension popup menu.

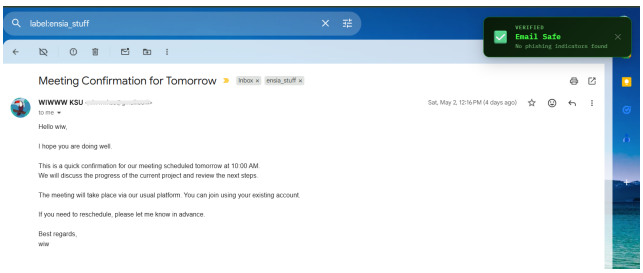

Fig. 12. PhishGuard browser extension displaying a real-time prediction inside Gmail.

IX. Discussion

A. Strengths

Strong, verified generalization. Test metrics match validation exactly (AUC 0.9985, F1 0.9825), and only 34 of 2,573 held-out emails were misclassified — confirming that the leakage-free experimental design produced a reliable, deployment-ready model.

Well-calibrated probabilities. The Brier score of 0.0109 confirms that confidence scores displayed to users accurately reflect true uncertainty, enabling informed human decisions on borderline cases.

Preprocessing robustness. Placeholder token substitution preserves the presence of URLs and numbers as signals without memorizing specific domains or values that would fail on unseen data.

Real-time speed. Sub-millisecond CPU inference makes browser-level detection practical without cloud infrastructure.

B. Limitations

The model is static and does not adapt to new phishing patterns post-training. The handcrafted lexicons are manually curated and may miss novel phishing vocabulary. The system analyzes only email text content, ignoring headers, sender reputation, and HTML structure. The local API deployment requires a running Python process, limiting accessibility for non-technical users.

C. Future Work

Periodic retraining on newly labeled data would keep the model current. Incorporating email header analysis (SPF/DKIM validation, sender domain age) would add non-textual signals. Fine-tuning a transformer model (BERT or RoBERTa) on email data could improve detection of conversational phishing — the primary failure mode identified. Cloud deployment would eliminate the local runtime requirement.

X. Conclusion

This paper presented PhishGuard, an end-to-end machine learning system for phishing email detection. A rigorous cleaning pipeline reduced 18,650 raw emails to 17,152 high-quality samples. A 8,022-dimensional feature representation combining structural heuristics, psycholinguistic lexicons, stylometric measures, and TF-IDF n-grams was constructed under strict zero-leakage discipline. A calibrated Linear SVM achieved a

test ROC-AUC of 0.9985 and **F1 of 0.9825** on 2,573 held-out samples — with only 34 misclassifications, a Brier score of 0.0109, and near-symmetric false positive/negative counts — outperforming ensemble methods and neural networks on this task.

Two key lessons emerge. First, **feature engineering quality is as important as model selection**: the gap between the handcrafted-only Random Forest (AUC 0.9454) and the full-feature Linear SVM (AUC 0.9985) shows that domain-aware representations are the primary performance driver. Second, **strict leakage prevention is essential**: the perfect consistency between validation and test metrics demonstrates that rigorous experimental design produces reliable, deployment-ready estimates.

REFERENCES

[1] I. Fette, N. Sadeh, and A. Tomasic, "Learning to detect phishing emails," in *Proc. 16th Int. Conf. World Wide Web (WWW)*, Banff, Canada, 2007, pp. 649–656.

[2] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Inf. Process. Manag.*, vol. 24, no. 5, pp. 513–523, 1988.

[3] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[4] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, San Francisco, CA, 2016, pp. 785–794.

[5] S. Ramirez, "FastAPI," 2019. [Online]. Available: https://fastapi.tiangolo.com

[6] C. Cortes and V. Vapnik, "Support-vector networks," *Mach. Learn.*, vol. 20, no. 3, pp. 273–297, 1995.